

UNIVERSITÉ D'ÉVRY PARIS-SACLAY

---

# Space Invaders

*Projet de programmation en langage C*

**RAIK Hamou & PANN Ilfan**

3<sup>ème</sup> année de Licence en Mathématiques et Applications  
Institut de Biologie, Génétique et Bio-informatique

Année universitaire : 2025 – 2026

# Table des matières

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                    | <b>2</b>  |
| 1.1      | Présentation du jeu . . . . .                          | 2         |
| 1.2      | Objectif du projet . . . . .                           | 2         |
| 1.3      | Compilation du code . . . . .                          | 2         |
| <b>2</b> | <b>Implémentation</b>                                  | <b>3</b>  |
| 2.1      | Affichage . . . . .                                    | 3         |
| 2.1.1    | Création des pixels arts . . . . .                     | 3         |
| 2.1.2    | Création d'une fenêtre graphique . . . . .             | 4         |
| 2.1.3    | Affichage des éléments graphiques . . . . .            | 4         |
| 2.2      | Déplacements . . . . .                                 | 7         |
| 2.2.1    | Déplacement du tank et de son tir . . . . .            | 7         |
| 2.2.2    | Déplacement des invaders et de leurs tirs . . . . .    | 7         |
| 2.2.3    | Déplacement de l'UFO . . . . .                         | 8         |
| 2.3      | Collisions . . . . .                                   | 9         |
| 2.3.1    | Collisions des bombes avec le tank . . . . .           | 9         |
| 2.3.2    | Collisions des tirs du tank avec les ennemis . . . . . | 10        |
| 2.4      | Etats de la partie . . . . .                           | 10        |
| 2.4.1    | Stockage des données . . . . .                         | 10        |
| 2.4.2    | Boucle de rendu . . . . .                              | 11        |
| 2.4.3    | Gestion des vagues d'ennemis . . . . .                 | 11        |
| 2.4.4    | Gestion du score . . . . .                             | 12        |
| 2.4.5    | Gestion des touches . . . . .                          | 12        |
| <b>3</b> | <b>Les problèmes rencontrés</b>                        | <b>13</b> |
| 3.1      | Ennemis se déplaçant trop rapidement . . . . .         | 13        |
| 3.2      | Sortie des invaders par le bas de la fenêtre . . . . . | 13        |
| 3.3      | Matrice d'envahisseurs considérée "pleine" . . . . .   | 14        |
| <b>4</b> | <b>Ajouts et améliorations possibles</b>               | <b>15</b> |
| <b>5</b> | <b>Sources</b>                                         | <b>16</b> |

# Chapitre 1

## Introduction

### 1.1 Présentation du jeu

Space Invaders est un jeu vidéo iconique développé par la société japonaise Taito Corporation et sorti en 1978 sur borne d'arcade. Un jeu qui est considéré comme le premier archétype du shoot them up, où l'objectif est de détruire des vagues d'aliens au moyen d'un canon laser en se déplaçant horizontalement sur l'écran. Conçu et programmé par Tomohiro Nishikado et qui s'est inspiré de plusieurs médias populaires de l'époque tels que Breakout ou La Guerre des mondes, il est aussi l'un des titres les plus influents de l'histoire du jeu vidéo.

### 1.2 Objectif du projet

Le but du projet est d'implémenter un Space Invaders, en respectant une partie des spécifications (non officielles) du jeu. Cela inclut :

- Affichage d'un menu.
- Défilement vers le bas des envahisseurs, sous la forme d'une matrice de 5 lignes et 11 colonnes.
- Déplacement du tank avec la souris.
- Gestion des tirs des envahisseurs (automatiquement) et du tank (avec l'appui sur la barre d'espace).
- Gestion du score
- Gestion de la rapidité de défilement des éléments graphiques (envahisseurs et tirs).

On utilisera la bibliothèque SDL2 pour la partie graphique.

### 1.3 Compilation du code

L'énoncé du projet impose que la compilation devra être faite avec la commande :

```
1 gcc -g -O2 -Wall -Wextra -o si *.c $(pkg-config --cflags --libs sdl2)
```

Comme dans notre implémentation, nous avons ajouté une fonctionnalité sonore en utilisant la bibliothèque SDL2\_mixer. Donc nous avons modifié la commande de compilation pour inclure cette dépendance :

```
1 gcc -g -O2 -Wall -Wextra -o si *.c $(pkg-config --cflags --libs sdl2  
    SDL2_mixer)
```

Ceci nous garantit l'intégration fluide des éléments sonores.

# Chapitre 2

## Implémentation

Le projet est constitué de plusieurs fichiers d'en-tête et source. Dans cette section nous allons décrire les principaux aspects techniques de l'implémentation du jeu *Space Invaders*.

### 2.1 Affichage

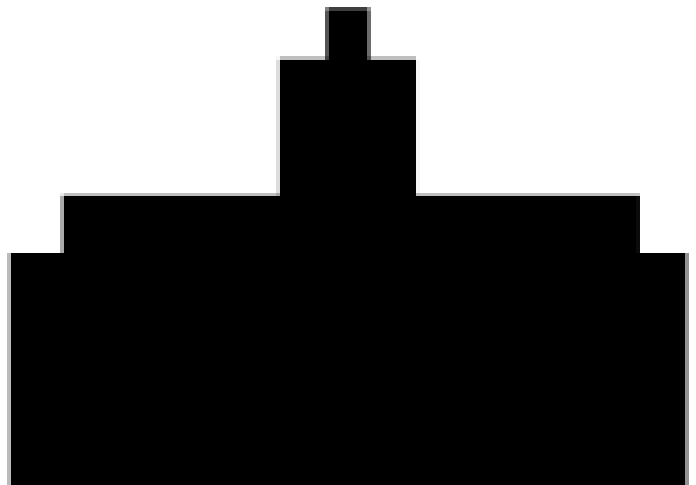
#### 2.1.1 Création des pixels arts

Dans le fichier `si_font.c`, les éléments graphiques du jeu sont représentés à l'aide de sprites déclarés et définis sous forme de tableaux binaires de type `char`, toutes de 8 lignes avec un nombre de colonnes variable et qui sont listés ci-dessous.

- *Tank* : représenté par `si_font_tank[8][13]` et pour son explosion on a `si_font_tank_explode[2][8][16]` qui contient 2 sprites dans des tableaux de 8 lignes et 16 colonnes et pour le tir il est représenté par un tableau vertical et stocké dans `si_font_tank_shoot[8][1]`.
- *Invaders* : sont regroupés en trois catégories (SQUID, CRAB et OCTOPUS), chacune disposant de deux modèles pour simuler une animation simple, et sont stockés dans `si_font_invaders[3][2][8][12]`, l'explosion des invaders et leurs tirs sont stockés dans `si_font_invader_explode[8][13]` et `si_font_invader_shoot[8][5]`
- *Ufo* : l'ennemi mystère qui passe de temps en temps stocké dans le tableau `si_font_ufo[8][16]`
- *Les caractères alphanumériques* sont au nombre de 43 et sont stockés dans un tableau à 3 dimensions `si_font_alphanum[43][8][5]`

Exemple :

```
1 char si_font_tank[8][13]={
2
3 {0,0,0,0,0,0,1,0,0,0,0,0,0},
4 {0,0,0,0,0,1,1,1,0,0,0,0,0},
5 {0,0,0,0,0,1,1,1,0,0,0,0,0},
6 {0,1,1,1,1,1,1,1,1,1,1,1,0},
7 {1,1,1,1,1,1,1,1,1,1,1,1,1},
8 {1,1,1,1,1,1,1,1,1,1,1,1,1},
9 {1,1,1,1,1,1,1,1,1,1,1,1,1},
10 {1,1,1,1,1,1,1,1,1,1,1,1,1}
11
12 };
```



### 2.1.2 Création d'une fenêtre graphique

La fenêtre est créée avec SDL. On a défini sa hauteur en fonction de la hauteur de notre écran personnel. Nous avons ensuite défini la largeur en fonction de la hauteur pour avoir le bon ratio. Ensuite une fonction SDL nous permet de récupérer les dimensions de l'écran pour pouvoir placer la fenêtre au centre

```

1
2  /* Récupère les dimensions de l'écran */
3  SDL_GetCurrentDisplayMode(0, &dm);
4  int x = dm.w/2 -273 +3;
5  int y = g->window_height/2 -360;
6  /* Création de la fenêtre */
7  g->win = SDL_CreateWindow("Space Invaders", x, y, g->window_width, g->
   window_height, SDL_WINDOW_SHOWN);
8  if (!g->win) return NULL;
9  /* Création du rendu associé à cette fenêtre */
10 g->ren= SDL_CreateRenderer(g->win, -1, SDL_RENDERER_ACCELERATED);
11 if (!g->ren) return NULL;

```

### 2.1.3 Affichage des éléments graphiques

Pour pouvoir afficher nos pixel arts à une taille bien visible, on parcourt la matrice d'un sprite et quand il y a un 1, on affiche le pixel comme un carré de côté 3.

```

1  void si_display_sprite(Game *g, char *data, int rows, int cols, int x,
   int y)
2  {
3      for (int w = 0; w < rows; w++) {
4          for (int h = 0; h < cols; h++) {
5              if (data[w * cols + h]) {
6                  SDL_Rect pixel = {
7                      x + h * g->pixel_size,
8                      y + w * g->pixel_size,
9                      g->pixel_size,
10                     g->pixel_size
11                 };
12                 SDL_RenderFillRect(g->ren, &pixel);
13             }
14         }
15     }
16 }

```

La fonction `si_display_sprite` permet ainsi d'afficher n'importe quel sprite juste en renseignant ses dimensions et ses coordonnées. On peut ensuite s'en servir dans une autre fonction `si_invaders_display`, qui affiche une vague d'ennemis en entier. Les envahisseurs sont représentés par une matrice `matrice[5][11]` dont chaque case contient :

- 0 : vide (ennemi éliminé),
- 1,2,3 : type d'ennemi (squid / crab / octopus) utilisé pour le score et le sprite.

```

1 void si_invaders_display(Game *g, int x0, int y0)
2 {
3     char *m = si_get_matrix();
4
5     /* Dimensions d un sprite invader : 8 lignes x 12 colonnes */

```

```

6      const int w = 12 * g->pixel_size;
7      const int h = 8 * g->pixel_size;
8      /* Espacement entre chacun des invaders */
9      const int gap_x = 2 * g->pixel_size;
10     const int gap_y = 5 * g->pixel_size;
11
12     for (int row = 0; row < 5; ++row) {
13         for (int col = 0; col < 11; ++col) {
14
15             int v = m[row * 11 + col];
16             if (v == 0) continue;
17             // 1,2,3 dans la matrice d'ennemis -> type 0,1,2 pour
18             si_invader_display
19             int type = 0;
20             if (v == 1)
21                 type = 0;
22             else if (v == 2)
23                 type = 1;
24             else if (v == 3)
25                 type = 2;
26             else continue; /* valeur inattendue */
27
28             int x = x0 + col * (w + gap_x);
29             int y = y0 + row * (h + gap_y);
30
31             si_invader_display(g, type, g->invader_frame, x, y);
32         }
33     }

```

Résultat obtenu :



Il y a aussi la fonction `si_text_display` qui permet d'écrire du texte, ici on ne passe pas par des coordonnées sur la fenêtre mais par des lignes et des colonnes. La taille de la fenêtre graphique a été découpée en 30 lignes et 26 colonnes, ses dimensions permettent d'écrire exactement 1 caractère par ligne et par colonnes.

```

1  /* Fonction qui affiche une phrase à l'écran */
2  void si_text_display(Game *g, const char *text, int ligne, int colonne, int
    spacing)
3  {
4      /* 1. On calcule la position de départ (Haut-Gauche) */
5      /* On suppose qu'une case de la grille fait 7 unités de large (5px de
        lettre + 2px d'espace) */
6      /* Et 8 unités de haut (hauteur de la lettre) */
7      int x = colonne * (7 * g->pixel_size);
8      int y = ligne * (8 * g->pixel_size);
9
10     /* 2. On parcourt le texte lettre par lettre */
11     for (int i = 0; text[i] != '\0'; i++)
12     {
13         /* On récupère l'index du caractère dans notre tableau de police */
14         int index = font_index_from_char(text[i]);
15
16         /* On affiche le sprite de la lettre */
17         /* 8 = hauteur du sprite, 5 = largeur du sprite */
18         si_display_sprite(g, &si_font_alphanum[index][0][0], 8, 5, x, y);
19
20         /* 3. On déplace le curseur X vers la droite pour la prochaine
            lettre */
21         /* Décalage = (Largeur de la lettre * taille du pixel) + espacement
            supplémentaire */
22         x += (5 * g->pixel_size) + spacing;
23     }
24 }

```

Ainsi pour pouvoir utiliser la fonction `si_text_display`, il faut d'abord définir une fonction `font_index_from_char` pour accéder aux matrices associées à chaque caractère du texte à afficher.

```

1  /* fonction qui à un caractère associe son index dans
2     si_font_alphanum définie dans si_font.c */
3  int font_index_from_char(char c)
4  {
5      /* on convertit les minuscules en majuscules */
6      if (c >= 'a' && c <= 'z')
7          c = c - 'a' + 'A';
8
9      if (c >= 'A' && c <= 'Z') return c - 'A';
10     if (c >= '0' && c <= '9') return 26 + (c - '0');
11     if (c == ' ') return 36;
12     if (c == '-') return 37;
13     if (c == '<') return 38;
14     if (c == '>') return 39;
15     if (c == '=') return 40;
16     if (c == '?') return 41;
17     if (c == '*') return 42;
18     /* si on a écrit un caractère non défini on affiche un espace */
19     return 36;
20 }

```

On a désormais un affichage fonctionnel d'images fixes, nous permettant de facilement afficher le menu, l'écran de jeu et l'écran de fin de jeu. Rajoutons ensuite des déplacements.

## 2.2 Déplacements

### 2.2.1 Déplacement du tank et de son tir

Le *Tank* est placé en bas de l'écran et peut se déplacer uniquement en horizontal, en suivant la position horizontale de la souris, dans le cas où une partie est en cours et que le jeu n'est pas en pause.

```

1      case SDL_MOUSEMOTION: //si la souris est déplacée
2          {
3              //on vérifie que la partie est en cours que le jeu n'est pas en pause
4              //et que le joueur possède au moins une vie
5              if (!g->paused && g->play_game && g->si->life_1 > 0) {
6                  //on met à jour la position du tanks avec la position horizontale de
7                  //la souris
8                  g->si->tank.x = events.motion.x;
9                  si_tank_set_position(g); // fonction qui met à jour la position du
10                 tank à l'écran
11                 g->update = 1;
12             }
13         }
14         break;
15     }

```

Le *tir du tank* se déplace verticalement. Il est initialisé sur la ligne située au-dessus du tank et centré horizontalement par rapport à celui-ci. Sa position est suivie à l'aide des variables `si->tank.shoot_x` et `si->tank.shoot_y`, et son déplacement est géré par la fonction suivante :

### 2.2.2 Déplacement des invaders et de leurs tirs

Les *Invaders* se déplacent dans deux sens, soit de droite à gauche ou de gauche à droite jusqu'à atteindre le bord de l'écran et ensuite descendent d'une ligne et vont dans le sens opposé ceci respectivement grâce aux fonctions :

- `si_invaders_can_move_left (Si *si)`
- `si_invaders_can_move_right (Si *si)`

**Exemple :** `si_invaders_can_move_right (Si *si)`

Renvoie 1 si les ennemis peuvent se déplacer vers la droite, 0 sinon et met à jour la position de la matrice.

```

1 int si_invaders_can_move_right(Si *si)
2 {
3     /*si les invaders vont à gauche on fait rien*/
4     if (si->invaders.direction != 1)
5         return 0;
6     int min_row, max_row, min_col, max_col;
7
8     if (!invaders_alive(&min_row, &max_row, &min_col, &max_col))
9         return 0;

```

```

10
11     int inv_w = 12 * si->pixel_size; //largeur des invaders
12     int gap_x = 2 * si->pixel_size; //Espace entre les invaders
13     /* colonne la plus à droite encore vivante */
14     int right_x = si->invaders.x + max_col * (inv_w + gap_x) + inv_w;
15
16     /*si cette colonne ne dépasse pas la largeur de la fenêtre on avance*/
17     if (right_x + si->pixel_size <= si->window_width)
18     {
19         si->invaders.x += si->pixel_size; //déplacement à droite
20         return 1;
21     }
22     /* on descend et on change de sens */
23     si->invaders.y += 8 * si->pixel_size; // descente verticale
24     si->invaders.direction = -1; // inversion du sens de déplacement
25     return 1; // mouvement effctué
26 }

```

Les bombes des invaders se déplacent aussi de manière verticale, une fois qu'on a choisi aléatoirement une colonne parmi les 11 de la matrice des ennemis grâce à la fonction, `si_invaders_get_column(Si *si)` on cherche l'invader le plus bas encore en vie et la bombe part en dessous de celui ci et le déplacement est géré par la fonction :

```

1     int si_invaders_bomb_can_move_down(Si *si)
2 {
3     /* si aucun invader n'est entrain de tirer il n'y a rien à déplacer*/
4     if (si->invaders.firing==0) return 0;
5
6     si->invaders.bomb_y += si->pixel_size; // déplacement vertical de la bombe
       ennemi
7
8     /*si la bombe sort de l'écran par le bas firing est remis à 0 */
9     if(si->invaders.bomb_y >= si->window_height){
10         si->invaders.firing=0;
11         return 1;
12     }
13
14     /*test de collision entre le rectangle du tank et celui de la bombe*/
15     if(si_tank_is_hit(si)==1)
16         return 1;
17     return 1; // la bombe continue sont déplacement
18 }

```

Pour finir, un minuteur est défini dans la fonction `game_run` (qui est la boucle principale du jeu) pour faire appel à ces fonctions de déplacements. Ce minuteur permet de gérer la fréquence à laquelle on déplace les invaders, ce qui permet de contrôler la vitesse de déplacement des ennemis.

### 2.2.3 Déplacement de l'UFO

L'*Ufo* se déplace soit de droite à gauche ou de gauche à droite jusqu'à atteindre le bord de l'écran puis disparaît ensuite. Ses déplacements sont gérés, comme pour les invaders, dans la fonction `game_run`. Chaque apparition de l'Ufo déclenche un cooldown de 15 secondes car c'est un ennemi qui fait des apparitions rarement. Une fois le cooldown terminé, pour qu'il fasse des apparitions aléatoires, on lui met 20% de chance d'apparaître chaque seconde.

```

1  /* APPARITION DE L'UFO */
2  Uint64 now = SDL_GetPerformanceCounter();
3  /* temps écoulé depuis le dernier test */
4  double dt = (double)(now - g->si->ufo.spawn_counter) / g->freq;
5
6  /* on attend 15 secondes avant d'essayer de spawn un ufo,
7   puis on tente un spawn chaque seconde
8   */
9  if (!g->si->ufo.active && now >= g->si->ufo.next_spawn_allowed && dt >
10  1.0) {
11      g->si->ufo.spawn_counter = now; // reset timer des tests
12
13      if ((rand() % 100) < 20) { // 20% de chance d'apparaître
14          g->si->ufo.active = 1;
15          g->si->ufo.y = 35 * g->pixel_size;
16          g->si->ufo.dir = (rand() % 2) ? 1 : -1;
17
18          int ufo_w = 16 * g->pixel_size;
19          g->si->ufo.x = (g->si->ufo.dir == 1) ? -ufo_w : g->window_width;
20
21          /* on définit le prochain spawn autorisé dans 15 secondes */
22          g->si->ufo.next_spawn_allowed = now + (Uint64)(15 * g->freq);
23
24          g->update = 1;
25      }
26  }

```

L'Ufo se déplace ensuite par la même méthode que les invaders, avec un minuteur, seule sa vitesse change car il est plus rapide.

## 2.3 Collisions

La détection des collisions dans le jeu repose sur une modélisation simplifiée des entités graphiques à l'aide de rectangles alignés sur les axes, appeler (*Axis-Aligned Bounding Boxes*). Chaque élément susceptible d'entrer en collision (tank, projectiles, envahisseurs) est associé à un rectangle défini par sa position à l'écran et ses dimensions.

Une collision est détectée lorsque les projections horizontales et verticales de deux rectangles se chevauchent simultanément.

### 2.3.1 Collisions des bombes avec le tank

Cette collision est détectée lorsque le rectangle représentant la bombe tiré par un invader intercepte le rectangle représentant le tank. En cas d'impact, le tank est marqué comme détruit, une vie est retirée au joueur et un effet sonore d'explosion est déclenché. Le tir ennemi est ensuite supprimé afin d'éviter toute collision multiple. La collision est gérée par la fonction suivante :

```

1  int si_tank_is_hit(Si *si)
2  {
3      /*si les invaders ne tirent pas le tank ne peut pas être touché*/
4      if(si->invaders.firing == 0) return 0;
5
6      /*coordonnées et dimensions du rectangle de la bombe des invaders*/
7      int bx = si->invaders.bomb_x;

```

```

8  int by = si->invaders.bomb_y;
9  int bw = 5 * si->pixel_size;
10 int bh = 8 * si->pixel_size;
11
12 /*coordonnées et dimensions du rectangle du tank*/
13 int tx = si->tank.x;
14 int ty = si->>window_height - (16 * si->pixel_size);
15 int tw = 13 * si->pixel_size;
16 int th = 8 * si->pixel_size;
17
18 /*test de collision avec l'intersection des 2 rectangles */
19 if (bx < tx + tw && bx + bw > tx && by < ty + th && by + bh > ty)
20 {
21     si->tank.destroyed = 1; //le tank est touché donc il est mit sur l'état de destruction
22     si->invaders.firing = 0; //arrêt du tir de l'invader
23     si->sound_explode_flag = 1; // déclenchement du son de l'explosion
24     return 1;
25 }
26 /*aucune collision détecté*/
27 return 0;
28 }

```

### 2.3.2 Collisions des tirs du tank avec les ennemis

Deux fonctions gèrent les collisions entre le tir du tank et les ennemis .:

**si\_invader\_is\_hit(Si \*si)** Teste si le tir touche un invader en parcourant la matrice des ennemis et vérifie l'intersection des rectangles correspondants. En cas de collision : l'invader est supprimé, le score est mis à jour, le meilleur score est éventuellement modifié et le son d'explosion est déclenché. La fonction retourne 1 si un invader est touché, 0 sinon.

**si\_ufo\_is\_hit(Si \*si)** Teste si le tir touche l'UFO. La collision est déterminée par l'intersection des rectangles du tir et de l'UFO. Si une intersection se produit : l'UFO est retiré, le tir arrêté, un nombre de points aléatoires (50, 100, 150 ou 300) est ajouté au score et un son d'explosion est déclenché. La fonction retourne le nombre de points obtenus si l'UFO est touché ou 0 sinon.

## 2.4 Etats de la partie

### 2.4.1 Stockage des données

La structure **Game** centralise la fenêtre SDL, le renderer, la configuration (dimensions, taille de pixel), les compteurs de temps. En plus de ce qui était attendu, nous avons rajouté certains états : pause, invader\_frame pour l'animation des ennemis, et tout une partie audio qui ajoute une musique et des effets sonores.

```

1 typedef struct
2 {
3     SDL_Window    *win;
4     SDL_Renderer  *ren;
5 }

```

```

6  int window_width;
7  int window_height;
8  int pixel_size;
9
10 int play_game;
11 int paused;
12 int invader_frame;
13 char update;
14
15 Si *si;
16
17 Uint64 freq;
18 Uint64 count_invaders;
19 Uint64 count_shoot;
20
21 /* Audio */
22 Mix_Music *bg_music;
23 Mix_Chunk *sfx_shoot;
24 Mix_Chunk *sfx_explode;
25 Mix_Chunk *sfx_wave;
26
27 } Game;

```

L'état du jeu (Si) contient les scores, vies, vague, ainsi que trois sous-structures : Tank, Invaders et Ufo contenant l'état de leur nom respectifs.

### 2.4.2 Boucle de rendu

La fonction `game_update` efface l'écran (fond noir), dessine les éléments (en blanc), puis appelle :

```
1 SDL_RenderPresent(g->ren);
```

L'affichage est déclenché uniquement quand `g->update == 1` afin d'éviter des rendus inutiles.

### 2.4.3 Gestion des vagues d'ennemis

Quand `si_matrix_count() == 0`, on passe à la vague suivante :

- incrément de `wave`,
- réinitialisation de la matrice d'ennemis et repositionnement du bloc d'envahisseurs,

```

1 void si_reset(Si *si)
2 {
3     /*réinitialisation des informations du jeu*/
4     si->score_1 = 0;
5     si->life_1 = 3;
6     si->wave = 1;
7     /*initialisation du tank*/
8     si->tank.x = (si->>window_width / 2) - (13 * si->pixel_size) / 2;
9     si->tank.firing = 0;
10    si->tank.destroyed = 0;
11    /*initialisation des invaders*/
12    si->invaders.x = si->pixel_size;
13    si->invaders.y = si->pixel_size * 60;
14    si->invaders.direction = 1;
15    si->invaders.firing = 0;

```

```

16  /*initialisation de l'ufo*/
17  si->ufo.active = 0;
18  si->ufo.spawn_counter = SDL_GetPerformanceCounter();
19  si->ufo.move_counter = SDL_GetPerformanceCounter();
20  si->ufo.next_spawn_allowed = SDL_GetPerformanceCounter();
21  /*direction de déplacement aléatoire*/
22  si->ufo.dir = (rand() % 2) ? 1 : -1;
23
24  /*réinitialisation des effet sonnores*/
25  si->sound_shoot_flag = 0;
26  si->sound_explode_flag = 0;
27  si->sound_wave_flag = 0;
28
29  /*réinitialisation de la matrice des invaders*/
30  for(int j=0; j<11; j++) matrice[0][j] = 1; //SQUID
31  for(int j=0; j<11; j++) matrice[1][j] = 2; //CRAB
32  for(int j=0; j<11; j++) matrice[2][j] = 2; //CRAB
33  for(int j=0; j<11; j++) matrice[3][j] = 3; //OCTOPUS
34  for(int j=0; j<11; j++) matrice[4][j] = 3; //OCTOPUS
35 }
36

```

— accélération via la formule de wave\_speed.

```

1  /* Accélération de la vitesse des invaders en fonction de la vague
   */
2  double wave_speed = 0.5 - ((g->si->wave) * 0.1);
3  if (wave_speed < 0.05) wave_speed = 0.05;
4

```

#### 2.4.4 Gestion du score

Le score gagné dépend du type d'ennemi (valeur stockée dans la matrice) :

- type 1 : 30 points
- type 2 : 20 points
- type 3 : 10 points

L'UFO donne un bonus aléatoire. Le hi-score (score le plus élevé) est chargé au démarrage depuis `highscore.txt`, puis mis à jour quand `score_1` dépasse `score_highest`. On réécrit alors le fichier.

#### 2.4.5 Gestion des touches

Certaines touches ont été implémentées avec SDL pour gérer l'état global du jeu :

- Echap ou q : quitter
- Espace : tirer en jeu ou relancer une partie depuis menu/game over
- s : stop / reprise du jeu
- Mouvements de souris (SDL\_MOUSEMOTION) : déplacer le tank horizontalement

# Chapitre 3

## Les problèmes rencontrés

### 3.1 Ennemis se déplaçant trop rapidement

**Explication :** Le déplacement des ennemis était beaucoup trop rapide lors de leur première implémentation, le problème ne venait pas de la fonction qui les déplace mais de la boucle de jeu `game_run` qui appelait la fonction de déplacement très vite.

**Solution trouvée :**

Nous avons utilisé le compteur haute-précision de SDL (`SDL_GetPerformanceCounter`) pour mesurer le temps entre deux mises à jour, puis calculé un delta-temps (`dt_invaders`) représentant le nombre de secondes écoulées depuis le dernier déplacement. Ce `dt_invaders` est ensuite comparé à une durée minimale (`wave_speed`) : Tant que `dt_invaders` est inférieur à `wave_speed` : aucun mouvement. Dès qu'il dépasse `wave_speed` les ennemis se déplacent une seule fois, puis le compteur est réinitialisé. Cela permet d'obtenir une vitesse indépendante des performances de l'ordinateur, stable peu importe la machine.

```
1 double dt_invaders = (double)(current_time - g->count_invaders) / g->
  freq;
2 if (dt_invaders > wave_speed)
3 {
4     /* ... (on gère les déplacements) */
5 }
```

### 3.2 Sortie des invaders par le bas de la fenêtre

**Explication :** Lors des premiers affichages, nous avons remarqué que lorsque les invaders atteignaient le bas de l'écran, ils continuaient à descendre jusqu'à sortir totalement de la fenêtre. Ils pouvaient alors s'entremêler avec le tank et passer en dessous de celui-ci. Une fois les ennemis sortis de l'écran, le jeu se retrouvait bloqué, car il n'y avait plus d'ennemis sur lesquels tirer et le joueur devras quitter et relancer une nouvelle partie.

**Solution trouvée :** Nous avons implémenter une fonction qui vérifie si le bas du groupe d'envahisseurs a atteint le haut du tank et on calcule la position du bas des envahisseurs et la compare à celle du tank. La fonction retourne 1 si une collision se produit et 0 sinon. Cela permet de déterminer si les envahisseurs ont progressé jusqu'à la zone défensive du joueur.

```
1 int si_check_collision_bottom(Si *si)
2 {
3     int min_row, max_row, min_col, max_col;
4     /*on vérifie s'il reste des invaders non détruits et on récupère leurs
   limites*/
```

```

5     if (!invaders_alive(&min_row, &max_row, &min_col, &max_col))
6         return 0; // plus d'invaders
7
8     int inv_h = 8 * si->pixel_size; // hauteur d'un invader
9     int gap_y = 3 * si->pixel_size; // espacement vertical
10
11     /* bas des envahisseurs */
12     int invaders_bottom =
13         si->invaders.y + max_row * (inv_h + gap_y) + inv_h;
14     /* position du haut du tank */
15     int tank_top = si->window_height - (16 * si->pixel_size);
16
17     return invaders_bottom >= tank_top; // renvoie 1 si les invaders on
18     atteint ou dépassé le tank
19 }

```

### 3.3 Matrice d'envahisseurs considérée “pleine”

**Explication :** Les invaders font demi-tour trop tôt lorsque des colonnes sont détruites. Le calcul prends en compte toutes les colonnes de la matrice des invaders pour savoir s'ils sont arrivés au bord de l'écran, même les colonnes vides d'ennemis. Résultats : les invaders font demi-tour trop tôt avant d'avoir atteint une extrémité de la fenêtre (gauche ou droite). Le même problème s'est posé aussi pour calculer si les invaders sont arrivés en bas au niveau du tank, les lignes vides d'ennemis sont prisent en compte dans le calcul des collisions. Résultats : collisions trop tôt.

**Solution trouvée :** Création d'une nouvelle fonction qui permet de revoyer les extrémités réelles de notre matrice d'ennemis, pour pouvoir ensuite faire les tests de collisions avec les bords.

```

1 static int invaders_alive(int *min_row, int *max_row, int *min_col, int *
    max_col)
2 {
3     int found = 0;
4     int rmin = 5, rmax = -1, cmin = 11, cmax = -1;
5
6     for (int r = 0; r < 5; r++) {
7         for (int c = 0; c < 11; c++) {
8             if (matrice[r][c] != 0) {
9                 found = 1;
10                if (r < rmin) rmin = r;
11                if (r > rmax) rmax = r;
12                if (c < cmin) cmin = c;
13                if (c > cmax) cmax = c;
14            }
15        }
16    }
17
18    if (!found) return 0;
19    *min_row = rmin; *max_row = rmax;
20    *min_col = cmin; *max_col = cmax;
21    return 1;
22 }

```

# Chapitre 4

## Ajouts et améliorations possibles

Malgré le fait que les instructions du projet donnent un jeu fonctionnel, nous avons fait le choix de rajouter certaines choses supplémentaires, comme la structure `Ufo` qui gère l'Ufo . De plus on a rajouté une touche pour mettre le jeu en pause en appuyant sur la touche `s`. Pour finir, l'ajout le plus important est celui de musiques et sons à l'aide la bibliothèque `<SDL2/SDL_mixer.h>`.

```
1  /* NOUVEAU : Init Mixer (Audio) */
2  if(Mix_OpenAudio(44100, MIX_DEFAULT_FORMAT, 2, 2048) < 0) {
3      printf("Erreur SDL_Mixer: %s\n", Mix_GetError());
4  }
5
6  /* Chargement des sons */
7  g->bg_music = Mix_LoadMUS("music.mp3");
8  g->sfx_shoot = Mix_LoadWAV("shoot.wav");
9  g->sfx_explode = Mix_LoadWAV("explosion.wav");
10 g->sfx_wave = Mix_LoadWAV("wave.wav");
11
12 /* Lancer la musique en boucle (-1) */
13 if(g->bg_music) Mix_PlayMusic(g->bg_music, -1);
```

Trois nouveaux membres dans la structure (`Si`) nous permettent de gérer les effets sonores des tirs, de l'explosion des ennemis et de l'arrivée d'une nouvelle vague d'ennemis.

```
1  int sound_shoot_flag; // bruit de tire
2  int sound_explode_flag; //bruit d'une explosion
3  int sound_wave_flag; // bruit lors d'une nouvelle vague
```

Ces variables vont jouer leur son respectif lorsqu'on met leur valeur à 1, sinon on les laisse à 0.

D'autre part, nous avons réfléchi à d'autres améliorations qui pourraient rendre le jeu encore meilleur :

- **Ajouter la possibilité d'un 2ème joueur** : Le jeu se limite à un seul joueur alors que tout est fait pour qu'on puisse jouer à deux.
- **Ajout de bunkers** : Le jeu original en contient, bien qu'ici ce n'était pas demandé, il serait tout de même intéressant de les implémenter.
- **Différents modes de jeu** L'ajout de modes supplémentaires comme le réglage du niveau de difficulté, un mode contre la montre pour détruire un maximum d'ennemis en un certain temps ou même l'ajout de nouveaux ennemis.
- **Couleurs** : Les graphismes pourraient être améliorés avec une variété de couleurs pour les ennemis ou le tank.
- **Musique et son** : On pourrait ajouter davantage de sons et de musiques.

# Chapitre 5

## Sources

1. **Police d'écriture Space Invaders** :  
<https://fontstruct.com/fontstructions/show/2454471/nishikado-extra>
2. **Musique de fond** : <https://www.youtube.com/watch?v=ITIFiLDpttw>
3. **Son du tir du tank** : <https://youtu.be/Dav5kn8BmY?si=iRGuS3WsRYhsMXjf>
4. **Son d'explosion (tank et ennemis)** : 8<sup>e</sup> son de la vidéo suivante :  
<https://www.youtube.com/watch?v=EA4h8l2zZ1gt=7s>
5. **Son de nouvelle vague d'ennemis** : <https://youtu.be/6ga4IICXyCE?si=vWYL3BCCtvF9lspG>
6. **Articles sur l'utilisation de l'audio avec SDL** :
  - [https://wiki.libsdl.org/SDL2\\_mixer/MixOpenAudio](https://wiki.libsdl.org/SDL2_mixer/MixOpenAudio)
  - <https://loka.developpez.com/tutoriel/sdl/sons/>